ChainCard
Transparent Donation DApp

SC6113 Individual Assignment - six-page draft
Prepared 11 October 2026. Public deployment and real-wallet verification are pending. This draft must be updated before submission.

1. Introduction

ChainCard is a single-campaign donation application designed to make contributions and organizer withdrawals publicly traceable. The prototype combines a Solidity contract, a MetaMask-enabled browser interface and a Flask backend. The intended public network is Ethereum Sepolia, using test ETH only. It does not collect real charitable donations.

2. Problem Statement

A donor viewing a conventional payment summary may have difficulty independently checking which contributions were received or when an organizer withdrew funds. A shared on-chain ledger can expose those movements without giving the application server custody of user wallets. However, visibility of a withdrawal does not establish how money was spent after it reached the organizer. The design makes that trust boundary explicit.

3. Objectives

The application should let a donor connect MetaMask, submit a positive donation, see its transaction hash and receipt status, and inspect recent confirmed records. The campaign organizer should be able to withdraw funds only to the organizer wallet. Invalid amounts, unauthorized withdrawal requests and wrong-network submissions should be rejected. The Flask backend should provide real contract data to the interface.

The scope is deliberately one campaign and one business contract. There are no tokens, voting, multiple campaign roles, refunds, lending or interest calculations. These features are not needed to demonstrate the required individual-assignment workflow.

Architecture and technology

4. System Architecture

Component	Responsibility
Browser UI + MetaMask	UI validates inputs. MetaMask authorizes the account and signs transactions. Transactions go through the wallet provider to the contract.
Flask read API	Serves static assets, contract configuration, chain state, recent events and transaction receipt status. It never receives keys or signs transactions.
Donation contract on Sepolia	Receives donations, restricts withdrawals, updates accounting and emits public events.
RPC provider / Etherscan	RPC exposes blockchain state to Flask. Explorer links let users independently inspect contract and transaction records.

Write path: Donor or organizer - browser form - MetaMask approval - blockchain contract - receipt.
Read path: Browser - Flask API - Sepolia RPC - state/events - browser history and totals.

The frontend and backend are served from one Flask service. This avoids separate API-origin configuration and simplifies Render deployment. There is no application database: successful donation and withdrawal events are the shared transaction record. Read-only visitors can inspect balances and history without connecting a wallet.

5. Technologies Used

Frontend: HTML, CSS and JavaScript, with a local ethers 6.15.0 bundle for ABI encoding and exact ETH/wei conversion. Backend: Python, Flask 3.1.2, Web3.py 7.13.0 and Gunicorn 23.0.0. Contract: Solidity 0.8.30, optimizer enabled with 200 runs, Shanghai EVM target. Development verification: Ganache 7.9.2, Node.js and Python unittest. Intended hosting: a Render Python Web Service and GitHub source repository.

The app is configured for chain ID 11155111. The backend checks the RPC chain and deployed code before reading the contract. The wallet network is checked before submitting a transaction. No private key is an application environment variable.

Contract and application design

6. Smart Contract Design

The deployer becomes the immutable organizer. totalDonated and totalWithdrawn record cumulative amounts in wei, and donatedBy stores each donor's cumulative contribution. donate() requires msg.value greater than zero, updates accounting and emits Donated. withdraw(amount) requires msg.sender to equal organizer and a positive amount no greater than the current balance. It updates withdrawal accounting, transfers ETH only to the organizer and emits Withdrawn. A failed transfer reverts the entire operation.

A shared reentrancy lock protects both state-changing entry points. The withdrawal function follows checks, then effects, then external interaction. It uses msg.sender for authorization. There are no loops over donors, limiting per-transaction work. Ordinary direct ETH transfers are rejected because no receive or fallback function exists. Forced ETH can still enter an EVM contract, so contract balance should not be assumed to equal totalDonated minus totalWithdrawn in every circumstance. [1]

7. Application Design

The interface has three sections: campaign totals and contract identity; wallet connection and donation/organizer withdrawal; public transaction history. The organizer withdrawal form is hidden for other accounts, while the contract enforces authorization independently. Inputs use decimal strings and BigInt/wei conversion instead of floating-point arithmetic. Positive values with up to 18 decimal places are supported.

Before sending, the app rechecks account and network, encodes the exact contract call and estimates Gas. Wallet rejection produces a cancellation message. After submission, the hash links to Sepolia Etherscan; the backend receipt determines success or failure. The browser retains a pending hash so a page reload can resume tracking. A timeout is shown as unconfirmed, never as success. [2]

The mobile layout stacks controls and wraps long addresses. Wallet connection requires an injected provider; mobile users should use MetaMask's built-in browser. A 390px browser layout check passed, but physical-phone wallet verification remains pending.

Implementation and controls

8. Implementation

The source is organized into contracts/Donation.sol, static/ HTML/CSS/JavaScript and compiled ABI/bytecode, and app.py. A reproducible compiler script generates Donation.json and copies the local ethers bundle with its license. Flask serves the frontend and exposes /api/config, /api/dashboard and /api/transactions/<hash>. /health reports process/configuration status; it is not proof of RPC availability.

The dashboard reads the organizer, cumulative amounts and balance at the selected latest block. Event reads are split into 500-block chunks within at most 5,000 blocks and sorted by block/log index; the UI shows the latest 30 events and discloses the query range. Older history remains on chain and is accessible using the contract explorer link. This bounded view avoids scanning the entire chain on every request, but public RPC limits can still make reads fail.

The receipt endpoint validates the hash, checks that the transaction targets the configured contract and returns pending, success or failed. Unknown transactions are treated as unconfirmed rather than successful. Backend failures return an error response. Browser rendering uses textContent for chain data, and a Content Security Policy restricts resources to the same origin. No server-side login is required because the server exposes public reads and cannot authorize a withdrawal.

Security, privacy and ethics

Wallet account access and transaction approval remain with MetaMask. Contract authorization is independent of the UI and server. The server does not store keys, recovery phrases or donor contact information. Wallet addresses, amounts and events are public; the UI warns users of this visibility and of the absence of refunds. A lost organizer key would prevent ordinary organizer withdrawals, and a compromised key could authorize them.

All transaction amounts are test ETH. The campaign is a classroom prototype, not an accredited charity. On-chain transparency does not verify the organizer's identity or off-chain use of funds. The code has local tests but has not received an independent security audit.

Deployment configuration

Render requires a Sepolia RPC URL, the deployed contract address and deployment block. These settings do not include secrets. The intended start command runs Gunicorn. No new public service or contract address has been verified for this version yet; the original ChainCard site is a separate, earlier wallet-query application.

Testing and results

9. Testing and Results

Executed local tests: the Solidity contract was compiled and deployed to an isolated Ganache EVM. Ganache used the Sepolia chain ID for compatibility; it was not the public Sepolia network. Flask integration tests queried that actual local contract. Frontend tests used simulated DOM, wallet and API responses. Evidence files distinguish these test environments.

Test group	Observed outcome
Contract: 10 checks	Deployer identity; zero donation; successful donation/accounting/event; non-organizer withdrawal; zero and excess withdrawal; successful withdrawal/balance/event; direct-transfer rejection; reentrancy rejection; failed-recipient accounting preservation. All passed.
Flask: 7 tests	Actual local contract state/events; successful receipt; invalid hash; pending hash; unconfigured deployment; RPC failure; static assets and security headers. All passed.
Frontend: 6 checks	Invalid amount/one-wei precision; rejected wallet request; network switch; wrong-network submission; non-organizer submission; donation confirmation flow. All passed with mocks.
Responsive browser check	390px viewport and 390px document width: no horizontal overflow. Screenshot shows the honest unconfigured-contract state.
Public-network acceptance	Not executed yet: real MetaMask connection, public contract deployment, successful Sepolia donation and withdrawal, deployed Render API and physical-phone operation.

Local evidence is saved in evidence/local-contract-tests.json, backend-tests.txt, frontend-tests.json and local-mobile-unconfigured.png. The successful local donation was 0.01 ETH and withdrawal was 0.004 ETH, leaving 0.006 ETH. These are simulated balances, not public-network transaction claims.

Before submission, attach real wallet connection and contract deployment screenshots; record public donation and withdrawal hashes; compare the UI history with Etherscan receipts; capture invalid-input/cancellation behavior and a real mobile view. Replace this draft status only after those checks pass. No video recording is required by the individual-assignment document.

Evaluation and conclusion

10. Challenges Encountered

The earlier ChainCard application read mainnet balances and signed plain messages. Those operations did not satisfy the requirement to execute Solidity business transactions. The revised implementation adds a small contract workflow and unifies it on Sepolia. Serving the interface with Flask reduces cross-origin deployment configuration. Local tooling needed a separate Python environment and a Solidity compiler; Ganache used its JavaScript fallback because native acceleration was unavailable on the installed Node version.

A previous GitHub browser upload was blocked by automatic approval review before this implementation. Publishing the prepared source may require a user-assisted upload if that restriction persists. No public deployment is claimed on the basis of local code or tests.

11. Limitations

The organizer retains discretionary withdrawal power and the app cannot verify downstream spending. There are no refunds, campaign deadlines, spending proposals or account recovery. Public RPC availability and history limits affect the UI; cold starts may delay free hosting. A receipt can be affected by a chain reorganization. Replaced or cancelled pending transactions may require manual wallet/explorer inspection; stopping local tracking does not cancel a transaction. Physical-phone and public-network validation are not complete.

12. Future Improvements

After completing the required public deployment and evidence, narrowly scoped improvements could include finalized-block confirmation indicators, paginated event indexing and verified organizer information. They are outside the current implementation. A production charity application would require legal/operational controls and an independent security review before accepting real funds.

13. Conclusion

The local prototype implements one complete donation/organizer-withdrawal workflow with Solidity authorization, wallet-signed transactions, a Flask read backend and a responsive interface. Executed tests cover important success and rejection paths. It is not yet a submission-ready final result: the remaining work is public Sepolia deployment, real MetaMask and Render acceptance evidence, and updating this report with verified addresses, hashes and screenshots.

References

[1] Solidity documentation. Security Considerations. https://docs.soliditylang.org/en/latest/security-considerations.html (accessed 11 October 2026).

[2] MetaMask documentation. Send EVM Transactions. https://docs.metamask.io/metamask-connect/evm/guides/send-transactions/ (accessed 11 October 2026).

[3] SC6113 Individual Assignment.docx, local course handout. Requirements, deliverables, testing and report sections. Group Project requirements excluded.